■ PackCheck Enterprise

Complete Project Structure, Codebase Directory & Engineering Architecture Specification

Target System: Esko AIForge Problem Statement 2 ('Ready to Launch?')	**Engineering Paradigm:** Zero-Hallucination Hybrid (Deterministic Math + Gemini 2.5 Vision)
Runtime Environment: Python 3.11 - 3.13 / Streamlit 1.40+ / PyMuPDF 1.25+	**Persistent Storage:** Local SQLite Database (packcheck.db)

1. Codebase Root Architecture & Directory Map

The repository is organized into distinct, decoupled tiers separating configuration, ingestion, deterministic checking, multimodal visual AI, persistent storage, interactive UI, and unit test automation. Below is the complete directory taxonomy:

Directory / File	Role & Engineering Responsibility
app/streamlit_app.py	Production user interface with 5 operational sections: Findings Explorer, Physical Review, Spec Cross Checker, Precedents, and AI Assista
src/packcheck/	Core engine package containing data models, PDF/Excel parsers, deterministic rules, SQLite storage, and LangGraph assistant.
data/packcheck.db	Local persistent SQLite database storing reviewer finding waivers, pressroom sign-offs, and custom physical parameters.
data/rules/	Statutory regulatory rulebooks in YAML/Markdown: FDA 21 CFR labeling, India FSSAI 2020 regulations, and corporate brand rules.
data/samples/	Repository of 11 real-world packaging test pairs (folding cartons, wrappers, cans, bottles) + pre-press meeting transcripts.
tests/	Comprehensive unit test suite with 28 automated tests across math, barcodes, dielines, FSSAI rules, and verdict policies.
.env / .env.example	Environment configuration for Google Gemini API keys, free-tier rate limits (5 RPM / 250 RPD), and offline fallback models.
pyproject.toml / requirements.txt	Package dependencies (pymupdf, streamlit, openpyxl, reportlab, langgraph, pillow, google-genai, pytest).

2. Top-Level Core Package: `src/packcheck/`

The `src/packcheck/` directory houses the primary engine modules orchestrating the verification pipeline:

Module Path	Key Classes / Functions	Detailed Description & Mechanics
src/packcheck/engine.py	run_compliance_check() ComplianceEngine	Central pipeline coordinator. Orchestrates artwork parsing, spec loading, rule selection (FDA vs FSSAI), ru
src/packcheck/models.py	ArtworkDocument, ApprovedSpecification, Finding, Evidence, NutritionTable, Rule	Strongly typed Pydantic V2 data models enforcing strict schema boundaries across extracted text blocks, v
src/packcheck/config.py	GEMINI_API_KEY, BASE_DIR, RULES_DIR, SAMPLES_DIR	Loads environment variables from root `.env` file via `dotenv`, sets up free-tier safe API rate limits, and exp
src/packcheck/verdict_policy.py	enforce_dual_quote_policy() deduplicate_findings()	Anti-hallucination gate. Requires verbatim string matches from both artwork PDF and approved Excel befo
src/packcheck/qa.py	ask_packaging_gemini_assistant()	Multimodal Q&A assistant using Gemini 2.5 Flash with grounded fallbacks. Ingests artwork, specs, transcri
src/packcheck/report.py	export_json(), export_csv(), export_html()	Generates certified pre-press artifacts: machine-readable JSON audit trails, tabular CSV findings, and stan

3. Ingestion & Dieline CAD Filtering Subsystem (`src/packcheck/ingest/`)

Packaging PDFs contain a mixture of printable consumer text and non-printing technical dieline guides. The `ingest` package cleanly separates these layers without raster distortion:

Module File	Core Functionality & Algorithmic Process
src/packcheck/ingest/artwork.py	Extracts vector text blocks, typeface point sizes, and bounding box coordinates using PyMuPDF (`fitz`). Renders high-resolution 200 DP
src/packcheck/ingest/dieline_filter.py	Pattern-matching and regex filter that purges technical CAD dieline annotations (e.g. 'CREASE', 'DIE CUT', 'GLUE FLAP', 'FOLD HERE'
src/packcheck/ingest/contentsheet.py	Parses approved master formulation Excel workbooks (.xlsx) using `openpyxl`. Extracts declared allergens, marketing claims, UPC/GTIN
src/packcheck/ingest/nutrition_parser.py	Deterministic text tokenizer extracting tabular Nutrition Facts panels: identifies calories, servings per container, metric gram values, and
src/packcheck/ingest/panels.py	Spatial panel segmentation algorithm dividing multi-panel carton dielines into Principal Display Panel (PDP), Information Panel, side pan

4. Vector Dieline CAD Geometry & Spatial Mechanics

Located in `src/packcheck/dieline/` and `src/packcheck/checks/vision/design_language.py`, this module measures physical package safety:

1. Euclidean Crease Margin Safety: Evaluates the perpendicular Euclidean distance from consumer copy vertices to dieline crease/score lines. Enforces a statutory safety margin of ≥ **8.0 pt (~2.82 mm)** to prevent copy cracking or folding into structural carton seams on high-speed automated folding lines.
2. Front Panel Title Centering: Computes the horizontal midpoint delta: $\Delta = |cx_{text} - cx_{panel}| / w_{panel} * 100\%$. Enforces $\Delta \leq 5.0\%$ tolerance to guarantee visual balance on retail display shelves.
3. Minimum Typeface Sizing: Inspects PDF font matrices to assert compliance with FDA 21 CFR § 101.2(c) and FSSAI rules (minimum 6.0 pt / 1/16-inch letter height for mandatory informational text).

5. Deterministic Compliance Engines (`src/packcheck/checks/deterministic/`)

This package eliminates LLM hallucinations by calculating mathematical, barcode, and formulation checks in pure Python:

Module File	Inspection Authority	Calculations & Regulatory Rules
nutrition_math.py	FDA 21 CFR § 101.9	Calculates % Daily Values against FDA Reference Daily Values (DRVs). Recalculates caloric bounds using Atwat
upc_check.py	GS1 General Specs § 7.9.1	Calculates Modulo-10 barcode check digits: (10 - ((3*Sum_odd + Sum_even) mod 10)) mod 10. Validates UPC-A
fssai_check.py	India FSSAI Regulations 2020	Validates 14-digit FSSAI license numbers (Central vs State codes), Veg/Non-Veg logo presence, Mfg/Best-Before
ingredient_diff.py	PLM Recipe Reconciliation	Cross-checks ingredient percentages (e.g. 8% approved honey vs 12% printed claim), flags unapproved ingredien
required_elements.py	FDA & FSSAI Statutes	Verifies presence of mandatory Statement of Identity, Net Quantity dual units (metric + imperial), manufacturer co

6. Persistent Storage & Audit Subsystem (`src/packcheck/db/storage.py`)

PackCheck maintains a local persistent SQLite database at `data/packcheck.db` tracking compliance history across production runs:

Table Name	Primary Columns	Functional Role in Pre-Press QA
finding_dismissals	finding_id, finding_title, category, rule_id, artwork_filename, reason, reviewer_name, timestamp	Stores reviewer manual sign-offs and regulatory waivers. Provides historical precedent retrieval w
physical_review_parameters	id, artwork_filename, param_name, spec_target, is_verified, checked_by, notes, timestamp	Stores custom physical packaging checks (e.g. Sutherland Rub Test, Hot Foil Stamping, Braille H
audit_log	id, action, artwork_filename, details, reviewer_name, timestamp	Immutable chronological system timeline tracking all scans, dismissals, restorations, and pressro

7. Pre-Press Transcript Intelligence (`src/packcheck/transcripts/`)

Parses pre-press meeting transcripts (`parser.py`) and executes BM25 semantic retrieval (`rag.py`) over alignment discussions. Crucially, it enforces the **Document Authority & Legal Precedence Hierarchy**:

- **Tier 1: Statutory Law** (FDA 21 CFR / India FSSAI)
- **Tier 2: Written Specification** (Master Content Sheet .xlsx v2.4)
- **Tier 3: Dieline Artwork Proof** (.pdf)
- **Tier 4: Meeting Transcripts** (Advisory Context Only - cannot legally override written specs)

8. Streamlit Production UI (`app/streamlit_app.py`) — The 5 Sections

The web dashboard is organized into exactly 5 consolidated production tabs post-investigation:

Section Name	Interactive Features & Engineering Capabilities
1. Findings Explorer	Displays all compliance findings with category filtering (Nutrition Math, Formulation, Claims, Regulatory, Design). Features the Root Cause
2. Physical Review	Left: Dieline proof preview with bounding box overlay and centering metrics. Right: AI-pulled rulebook checks (minimum 6pt font size, 8pt cr
3. Specification Cross Checker	Side-by-side comparative reconciliation between approved PLM specifications (.xlsx) and printed dieline copy (.pdf). Highlights recipe mism
4. Precedent Logs of All Kinds	Unified repository with a Global Search Bar and 4 sub-tabs: (1) Reviewer Dismissals, (2) Physical Review & Sign-Offs, (3) Meeting Transcri
5. AI Packaging Assistant	Interactive conversational assistant powered by Google Gemini 2.5 Flash and grounded reasoning. Ingests full product context: specs, artw

9. Regulatory Rulebooks Repository (`data/rules/`)

Regulatory standards are defined declaratively in machine-readable YAML files loaded into Pydantic models:

Rulebook File	Jurisdiction / Scope	Defined Rules & Validation Logic
data/rules/fda_food_labeling.yaml	US FDA 21 CFR § 101	Mandatory Statement of Identity (PDP), Net Quantity dual units, Nutrition Facts panel format, 21 CFR 1
data/rules/fssai_food_labeling.yaml	India FSSAI Regulations 2020	14-digit FSSAI License Number format (`FSSAI-LIC-001`), Veg/Non-Veg logo geometry (`FSSAI-VEG-0
data/rules/company_rulebook.md	Corporate Brand Guidelines	Brand typography standards, minimum logo safety clearances, certified vegan restrictions against hone

10. Automated Unit Test Suite (`tests/`) — 28/28 Passing

PackCheck maintains 100% pass rates across 28 unit tests verifying every mathematical formula, dieline filter, and regulatory check:

Test Module	Tests	What It Mathematically & Operationally Verifies
test_nutrition_math.py	5	Verifies %DV calculation against FDA DRVs, Atwater 4-4-9 caloric bounds, and fat/carb subcomponents.
test_fssai.py	3	Tests FSSAI 14-digit license regex, statutory 2% trans-fat cap enforcement, and valid/invalid passes.
test_upc.py	4	Tests GS1 Modulo-10 checksum algorithm, valid/invalid check digits, and length constraints.
test_ingredient_diff.py	3	Verifies recipe percentage mismatches (Honey 8% vs 12%), unapproved walnuts, and duplicate seeds.
test_dieline_filter.py	2	Asserts technical CAD guides are stripped while actual packaging copy is preserved.
test_meeting_transcripts.py	3	Verifies speaker turn parsing, RAG semantic search, and Written Spec Precedence enforcement.
test_verdict_policy.py	4	Tests anti-hallucination dual-quote policy (downgrading unverified LLM claims) and deduplication.
test_artwork_ingest.py	1	Tests PyMuPDF vector text extraction, font size calculations, and high-res rendering.
test_contentsheet.py	1	Tests openpyxl ingestion of master formulation Excel sheets and metadata mapping.
test_e2e_golden.py	1	End-to-end golden run on DancingFish Granola verifying comprehensive finding generation.
test_false_alarm.py	1	Verifies that a 100% clean compliant packaging sample produces zero false alarms.

11. Packaging Benchmark Samples (`data/samples/`)

The repository includes 11 complete packaging pairs for evaluation: 1 Golden Sample (DancingFish Granola) + 10 un-tested FMCG benchmark pairs across folding cartons, flow wraps, glass bottles, metal cans, and dairy tubs.